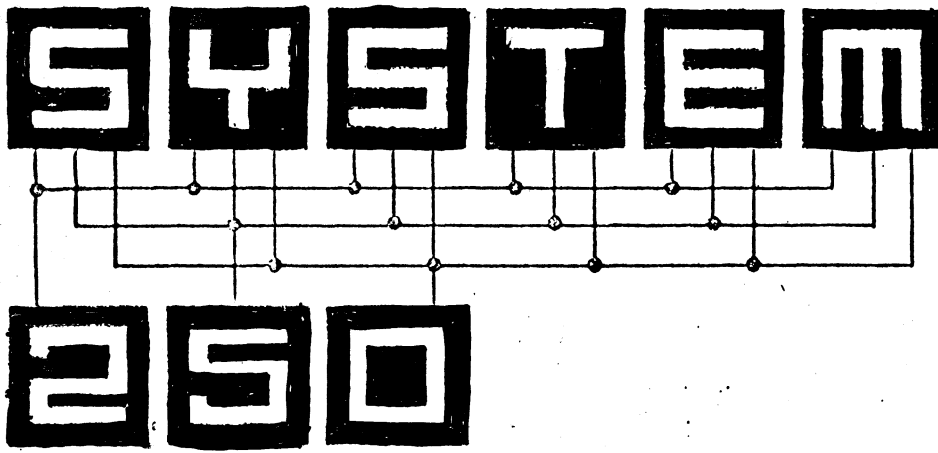




General Introduction

© 1972 The Plessey Company Ltd.

The information contained herein is the property of The Plessey Company Limited, and is not to be disclosed or used either in whole or in part without the prior written permission of The Plessey Company Limited.

The right is reserved to make any modifications to the information herein without prior notice.

Plessey Telecommunications Ltd.,
Edge Lane,
Liverpool L7 9NW.

Ref. SPC1
April 1972

CONTENTS

PHOTOGRAPHS

Commissioning the Store Access Unit on a single Processor/single Store system.

Software commissioning using a three Processor/three Store system and associated test equipment.

	Page
1. INTRODUCTION	1
1.1. General	1
1.2. Benefits of the SYSTEM 250	1
2. SYSTEM 250 HARDWARE	3
2.1. Introduction	3
2.2. PP250 Processor	3
2.3. Series 250 Store	4
2.4. Store Access Unit	5
2.5. Bus Multiplexor	5
2.6. Parallel Interface Unit	6
2.7. Serial Medium	6
2.8. Serial Interface Unit	6
2.9. Input/Output	7
2.10 Serial Medium Transfers	7
3. SYSTEM 250 OPERATING SYSTEM	8
3.1. Introduction	8
3.2. Capabilities	9
3.3. 'Virtual Machine' Packages	10
3.3.1. <i>Fault Security</i>	10
3.3.2. <i>Store Manager</i>	11
3.3.3. <i>Process Manager</i>	11

	Page
3.3.4. <i>Input/Output Manager</i>	11
3.3.5. <i>File Manager</i>	12
3.3.6. <i>Directory Manager</i>	12
3.4. <i>Man/Machine Communication Packages</i>	13
3.4.1. <i>Command Interpreter</i>	13
3.4.2. <i>Command Definition and Assembly</i>	13
3.4.3. <i>Job Control</i>	14
3.4.4. <i>Directory Control</i>	14
3.4.5. <i>File Editing</i>	14
3.4.6. <i>Store and Process Commands</i>	14

FIGURES

1. Basic System
2. Small Secure System
3. Medium System Configuration.

THE PLESSEY SYSTEM 250

GENERAL INTRODUCTION

INTRODUCTION

1.1. General

The Plessey SYSTEM 250 has been designed with particular reference to the requirements for controlling telephone and data exchanges (both analogue and digital). It is based on Plessey's experience in designing and producing telephone exchanges, and on their experience in complex real-time computer installations such as air-traffic control, message switching, traffic signalling systems, etc. The experience gained from the earlier Plessey XL series of computers was also drawn upon during the specification and design stage.

1.2. Benefits of the SYSTEM 250

In the early design stages a combined analysis was made of the telephone application area and of the Company's experience in other real-time areas and this was then compared against the results of the three Processor exchange model built at Taplow in 1969. The main requirements were seen to be:-

- (1) flexibility in design to meet a wide range of applications;
- (2) economic use of high-cost modules, e.g. storage;
- (3) enduring system design philosophy;
- (4) security of service against system faults;
- (5) ability to expand while in service.

The System designed as a result of this analysis is truly modular and multi-Processor, with only two types of standard interface. It can be configured in terms of processing power, storage and Input/Output to meet the requirements of the particular application, and extra modules can be added on-line with a minimum of effort to handle growth of traffic or new facilities.

The security mechanisms built into the hardware of SYSTEM 250 not only recognise and isolate faults with necessary reconfiguration, but also enable the sharing of program store and time-sharing of both real-time and batch programs, including program development, without the chance of corruption of proven modules from errors elsewhere.

The Operating System is designed on a modular basis with clear interfaces to the application modules. It will provide an environment for the programmer that is transparent to any actual hardware configuration and this fact combined with a simple and efficient instruction set makes the System easy to program.

2. SYSTEM 250 HARDWARE

2.1. Introduction

The principal component modules of SYSTEM 250 are discussed in outline below. Figs. 1 to 3 illustrate how these components are linked together in typical system configurations.

2.2. PP250 Processor

The PP250 PROCESSOR is the Central Processing Unit of SYSTEM 250 and has been designed as a system module. As such it may form part of a wide range of system configurations, from a simple one-Store/one-Processor system to a multi-Store/multi-Processor system. As the expansion of the processing system is independent of that of the storage and Input/Output systems, a configuration may be built in which the installed processing power and storage capacity closely match the requirements of any particular situation.

The Processor is a 24 bit word length, one address machine in which all data transfers and manipulations are performed in parallel mode. The machine is equipped with CAPABILITY protection measures (see Sect. 3.2.) which prevent single hardware or software defects causing access to wrong areas of store. Although a program has potential access to any area of storage, a Capability structure, superimposed by software and maintained by hardware, permits the software to predefine not only those regions of store which a particular program may use, but also the way in which they may be used. Any defect, hardware or software, which causes an attempted violation of previously imposed Capability constraints causes an automatic fault sequence to be entered.

The Processor has two major types of register. There are eight general purpose Capability Registers into which the software can load the boundaries of store blocks to which a process has access. Each of these Capability Registers is also loaded with a code which defines the type of access allowed to the process within a specified store block. There are also eight general purpose Data Registers, in which all data manipulation is performed.

The Processor is structured in a way which permits process orientated software strategies to be employed. A process is defined as the application of a block of code (embodied in program) to an external event (as described by input data).

The Processor is equipped with 25 basic instructions which are orientated towards the requirements of the Operating System software, particularly with respect to process control and event processing applications, and to the operation and manipulation of Capabilities.

To maintain the multi-Processor traffic-sharing philosophy of SYSTEM 250, and avoid the problem of connecting dedicated interrupt lines to Processors (and the consequent problem of expansion), the Parallel Interface Units (see Section 2.6) are polled on a regular basis and serviced should they need attention by a process which may be run on any of the Processors in a given system. This ensures that faults occurring in the Input/Output network do not significantly affect the operation of any Processor, and vice versa.

2.3. Series 250 Store

The main storage module being supplied is the Plessey Series 250 Store. Each module is a self-contained unit with its own dedicated power supplies and test facilities and normally contains 32K words of 25 bits (one bit parity).

The module consists of a plated wire array having an approximately 300 ns Read or Write cycle time with decode, drive and sense electronics, providing low power, non-volatile storage. Data protection facilities are fitted in order to ensure no loss of data during power supply interruptions or failures. It is relatively insensitive to environmental temperature changes and will operate satisfactorily in an electro-magnetically noisy environment.

Any type of store which provides 25 bit store locations, such as core store, may be used as well as, or in place of, Series 250 plated-wire modules. At present Planar 850 core store, which is available in modules ranging from 8K to 32K words, is also being interfaced to the System. This has a cycle time of 850 ns.

2.4. Store Access Unit

The STORE ACCESS UNIT is a device which is intimately connected to a Store module. It possesses a number of ports by which Processors can gain access to the Store module over their own individual buses.

The basic Store Access Unit is provided with 4 ports and is designed to be easily expandable to provide up to 8 ports within a single module. Conversely, it can be depopulated to provide 1, 2 or 3 ports. An extra module is required for Store Access Units with more than 8 ports, but the same design philosophy and interface boards are used.

The Store Access Unit performs the following principal functions:-

- (1) It recognises the address presented on any one of the ports, which corresponds to the Store module number associated with the Store Access Unit;
- (2) It allocates successive store access cycles to Store demands in the order prescribed by the priority of the port, unless a special control signal requires the Store to be locked to a particular port;
- (3) It generates the parity of both the address and data which it receives and returns this on a parity check wire to the initiating end where it can be compared.

If the Store is free when a module address demand is recognised, the demand is allocated immediate access. If the Store is busy on another access, demands on other ports are held until they are given access in order of priority. No acknowledgement is made to the initiating end of a demand until that demand has been allocated a store cycle.

2.5. Bus Multiplexor

In small systems peripherals may be attached directly to the main Parallel Processor Buses as in Fig. 1, but in larger systems with more than two Processor Buses, this would be too complex. Instead, peripherals are attached to two separate parallel peripheral buses (Fig. 3). The BUS MULTIPLEXOR handles exchanges of data between a number of Processor Buses and a single Peripheral Bus.

The Multiplexor performs an address recognition function in a similar manner to the Store Access Unit in that it examines the most significant bits in an address for comparison with its module number (pluggable to include any of the most significant 16 bits). If the comparison is favourable the Multiplexor allows the full 24 bit address to pass through to the associated Peripheral Bus.

To secure access to peripheral devices in the event of a failure, Multiplexors, and thus also peripheral buses, are normally duplicated.

2.6. Parallel Interface Unit

This unit is used to interface a peripheral device directly to the Processor or Peripheral Bus(es). It is particularly suited to peripheral devices which are frequently used and operate at high data transfer rates (e.g. drums, discs, magnetic tape units). Each Parallel Interface Unit is particular to the type of peripheral it is interfacing to the system. The use of Parallel Interface Units ensures that many different types of peripherals all present the same standard interface to the System.

2.7. Serial Medium

To provide the ability to communicate with a large number of low activity peripheral devices, including telephone or data devices, SYSTEM 250 features a serial data collection and distribution medium which is interfaced to the Peripheral Bus(es) through a SERIAL-PARALLEL ADAPTOR. This SERIAL MEDIUM consists of a network of DATA SWITCHES of two types: 64 way PRIMARY SWITCHES and 16 way SECONDARY SWITCHES to which low activity peripherals are interfaced through SERIAL INTERFACE UNITS. In relation to the Parallel Bus System the Serial Medium containing many Data Switches and Serial Interface Units is treated as a special type of high activity peripheral and thus the Serial-Parallel Adaptor is treated as a Parallel Interface Unit.

2.8. Serial Interface Unit

Low speed peripherals such as paper tape readers, teleprinters, as well as telephone and data communication equipment, which do not require the high data transfer rate potential of the Parallel Bus System are serially interfaced to the SYSTEM 250 Serial Medium through Serial Interface Units. The Serial Interface Unit ensures that different types of peripherals all present the same standard interface to the Serial Medium.

2.9. Input/Output

Devices which interface directly to the Peripheral Buses (viz. Parallel Interface Units) have much in common with Store Access Units, but are simplified to handle a maximum of two buses. Each of such devices looks to a Processor like a block of store containing a small number of locations which are in fact the Command, Status and Data registers of the device.

In view of the above, no special Input/Output instructions are used in SYSTEM 250, the general instructions LOAD DATA and STORE DATA being sufficient to communicate with the registers in the appropriate device. The treatment of all data sources and destinations as though they were store locations greatly simplifies the handling of Input and Output transfers. This approach also enables all data transfers to and from peripherals to be subject to the Capability protection scheme. In addition, the allocation and use of peripheral registers is controlled by the Supervisor in exactly the same way as for blocks of store.

2.10. Serial Medium Transfers

Data transfers between the peripherals and the Serial-Parallel Adaptor take place automatically. The data is in the form of outgoing (or incoming) messages up to 32 bits in length which are distributed from (or collected in) an output (or input) buffer in the Serial-Parallel Adaptor. In order to specify its source or destination, each data message has attached to it an identifying code consisting of an address and Check Code particular to one peripheral device. The address is used to route the message through the Data Switch network, and the Check Code is used to check the validity of the routing when the message reaches its destination.

The Serial-Parallel Adaptor has its input buffer emptied and its output buffer filled by polling programs which are executed at suitable intervals. To secure access to serial peripherals in the event of a Data Switch or Serial-Parallel Adaptor failure, an alternative path to each device is normally provided by duplication of the Serial-Parallel Adaptor and Serial Medium.

SYSTEM 250 OPERATING SYSTEM

3.1. Introduction

The SYSTEM 250 Operating System is a group of program packages which provide a wide range of facilities for the user of SYSTEM 250 hardware. Each package has been designed as an independent entity so that only those required need be used, without overheads being incurred due to the remainder.

SYSTEM 250 has been designed so that a configuration can be made available which contains the minimum facilities, but with the ability to grow in several stages until it uses the complete SYSTEM 250 Operating System. The minimum configuration would consist of those hardware modules required for reasons of fault security, storage and processing power, together with the Fault Security, Store Management and Process Management software packages. The Input/Output hardware required by a particular configuration would be accompanied by the appropriate Input/Output software packages.

All Operating System packages are accessed by means of subroutine calls which use the protected "Enter" mechanism provided as an integral part of the PP250 Processor. Each package has access to those data areas which it requires in order to carry out its function, but not to any other data areas either inside or outside the Operating System. This means that the traditional distinction between 'privileged' Operating System functions and 'unprivileged' application program functions is entirely absent - instead, each package has exactly the access rights that it requires and no more. The consequence of this system organisation is that:

- (1) it is possible to protect the system more closely against software and hardware failures which might otherwise violate permitted storage boundaries.
- (2) it is possible to modularise the Operating System into a number of packages giving distinct facilities.

Below, after an introduction to the Capability concept, are described the principal software packages which comprise the SYSTEM 250 Operating System. These packages fall into two groups. The first group enhance the operation of the processing hardware itself by extending the facilities available to the programmer. Without this group of packages,

the programmer would have at his disposal only the instructions supported by the PP250 Processor hardware. Each function provided by each package may therefore be regarded as a 'virtual instruction' so that the programmer is presented with much more powerful programming facilities than would be provided by the hardware alone. The PP250 Processor, together with this group of packages, may therefore be regarded as a 'virtual machine' containing a powerful instruction set. The second group of Operating System packages provide the PP250 'virtual machine' with comprehensive man/machine communication facilities which permit the system to be used in a variety of applications where man/machine communication is a central requirements.

3.2. Capabilities

The Processor and Operating System use CAPABILITIES to exercise control over all of the processes that can be run within a given complex of Processors, Stores and Input/Output devices. This control is necessary since any of the stored processes could be called upon to run at any instant due to an internal or external stimulus to fulfil speed, efficiency and operational requirements. The process must be able to proceed without any possibility of errors, due to other processes being in the partially-run state (i.e. interrupted) or the process itself inserting errors in these latter processes. A further very important consideration is that since the data (which is applicable to the environment that the system is controlling) is also stored in the complex, then this data must not be altered in any way except by the processes which have permission to do so. In this way information (programs or data) is protected from illegal use, since any attempt to gain access to a block of store or area of the Input/Output system for which permission has not been given causes a Fault Interrupt. Hence any hardware or software error which can so affect the system is detected before serious damage occurs.

Processes are only allowed access to blocks of store for which they have been allocated Capabilities by the Operating System. Each Capability consists of three fundamental constituents, these being:

- (1) the BASE address or start of the block covered by the Capability,
- (2) the LIMIT address or end of the block,
- (3) the ACCESS TYPE which defines the operations allowed within the block, e.g. reading, writing etc.,

The size of a block of store covered by a Capability can be any number of words between one and 64K. The upper limit on size arises from the hardware convention that a block should be wholly contained within a single Store module. Store modules will normally contain 32K words but may be reduced to any requisite size, or expanded up to a maximum of 64K words per module. It is possible to have up 256 such modules in a system.

3.3. 'Virtual Machine' Packages

The SYSTEM 250 Operating System packages which enhance the hardware facilities of the Processor by effectively extending its instruction set, are as follows:-

3.3.1. Fault Security

The inclusion of the FAULT SECURITY packages in a SYSTEM 250 configuration permits a grade of system reliability very much higher than the inherent reliability of each of the hardware modules of which it is composed.

The Fault Security programs have been designed to reflect and to augment the features of the SYSTEM 250 hardware concerned with the detection and correction of faults. Therefore, the handling facilities must be thought of as a combined hardware and software mechanism, designed to provide secure system configurations, intended to give uninterrupted service for long periods even though individual hardware and software failure will occur during that time.

The detection of hardware failures, both transient and permanent, is a fundamental feature of the Fault Security System. However, it is equally important for the SYSTEM 250 to withstand the effect of software failure (program errors). It is unrealistic to assume that a software system of any size will be free of program "bugs" even after several years of service. The fault handling mechanisms are therefore designed with the aim of continuing this kind of fault so that the system can continue to run, in the face of the occasional software failure, without experiencing a catastrophic and permanent degradation in performance.

3.3.2. Store Manager

The STORE MANAGER package provides facilities for dynamic allocation and de-allocation of blocks of store, for testing and changing the length of blocks, and for reducing the permitted access conditions to blocks.

Each block of store is defined in SYSTEM 250 as a contiguous region within a physical storage module bounded by the Base and Limit values of a referencing Capability. The only way of accessing store is by means of Capabilities and therefore it is possible to arbitrarily restrict the storage regions which can be accessed from a given package by providing, in the package Capability Block, just those Capabilities required for the operation of that package.

3.3.3. Process Manager

The PROCESS MANAGER package provides facilities for the creation and deletion of processes, and for the creation and deletion of FLAGS. Flags are SYSTEM 250 resources which permit synchronisation between processes. Two or more processes may communicate through a Flag, and the Process Manager will suspend processes if they are blocked awaiting communication from another process via a Flag. In this way, the package provides the basic facilities required by the programmer for inter-process communication and control.

Timing Facilities are provided to enable processes to suspend awaiting a given time interval, and the Flag and timing facilities can be used together to enable a process to suspend awaiting either one of a number of Flags or a time interval. The process is rescheduled when one of the eventualities awaited occurs. This facility permits a very simple solution to a number of real time application software problems where an alternative course of action must be taken depending on (for example) whether or not a message arrives within a given time interval.

3.3.4. Input/Output Manager

The INPUT/OUTPUT MANAGER consists of a series of packages providing facilities for the control of input and output devices. The devices it handles are those connected to the parallel and serial Input/Output media and one package is provided per device type.

The basic Input/Output package normally permits the allocation of an Input or an Output Stream which is a device-independent symbolically named entity which is tied to a particular device by a Job Control command (see below) just prior to running the program. The purpose of this procedure is to enable programs to be written in a device-independent manner so that they may be run subsequently in different environments.

Input/Output Streams, Input/Output devices, and sequentially accessed files (see below) are interchangeable resources in SYSTEM 250. Exactly the same macro-instructions are used for transferring information to and from these resources.

3.3.5. File Manager

The FILE MANAGER package permits the allocation and de-allocation of files, in which text may be stored, by means of standard subroutine calls (macro-instructions). Files are freely extendible and are intended to hold information either of a sequential nature (Input/Output Streams) or of a random nature (lines of text).

In those installations requiring extensive use of file management facilities, backing storage devices (such as fixed head discs and drums) must be included in the system configuration. In these circumstances, the file management routines will also handle the transfer of blocks to and from the backing store.

3.3.6. Directory Manager

The DIRECTORY MANAGER package permits the allocation and de-allocation of a data structure known as a Directory. A Directory is a list of symbolic names against which are stored Capabilities (and/or data values). The purpose of a Directory is to achieve a mapping between the external names of resources (symbols) and the internal names of resources (Capabilities).

Standard subroutine calls enable the Directory to be searched for a given entry, and they enable new entries to be inserted and existing entries to be deleted. The Command Interpreter and Directory Control software (see below) make extensive use of the basic Directory data structure allocated by the 'virtual machine'.

3.4. Man/Machine Communication Packages

The 'virtual machine' comprising the packages described above together with the System hardware, provides the basic facilities on which all other program packages depend. These packages include both the second group of packages within the SYSTEM 250 Operating System (the MAN/MACHINE COMMUNICATION PACKAGES) and all application packages written to run on the system. The man/machine communication packages extend the facilities beyond those provided by the 'virtual machine' as described below:-

3.4.1. Command Interpreter

The COMMAND INTERPRETER provides the basic facilities for communication between man and machine. The communication may take place via any suitable character-by-character interface either in an interactive environment (e.g. via teleprinter) or in a non-interactive environment (e.g. via paper tape). The command language accepted by the Command Interpreter is in all cases identical.

The Command Interpreter requires access to one or more command libraries in order to function as a man/machine interface. The Command Interpreter carries out the interpretation of commands which appear on the Input Stream by selecting the relevant Command Program from a COMMAND LIBRARY. It is the execution of this Command Program which causes the effect required as the result of the appearance of the command on the Input Stream.

The Command Interpreter provides open ended facilities to the application programmer, who is able to design and specify his own Command Programs and his own libraries of commands as appropriate to a particular installation. The number of libraries supplied may be large or small so that there is no system overhead due to the presence of unwanted packages in small system configurations.

3.4.2. Command Definition and Assembly

This Command Library provides a series of commands with which the user can, on the one hand, develop his own application-oriented Command Libraries, and on the other hand, assemble the blocks which will comprise application program packages. This latter facility permits the Command Interpreter to be used as a macro-assembler which accepts as source both SYSTEM 250 Assembly Language statements and also user defined and system defined macro-instructions.

This Command Library permits the user of the system to control the progress of a particular 'job' or 'application'. This term should be interpreted widely to include, for example, the control of a real-time application, the control of an individual application programmer's job, or operator control of the central processing equipment.

The basic facilities provided are those for the allocation and de-allocation of Input/Output devices and for establishing and running jobs. It also monitors the user's access to the system by the use of identity names and passwords.

3.4.4. Directory Control

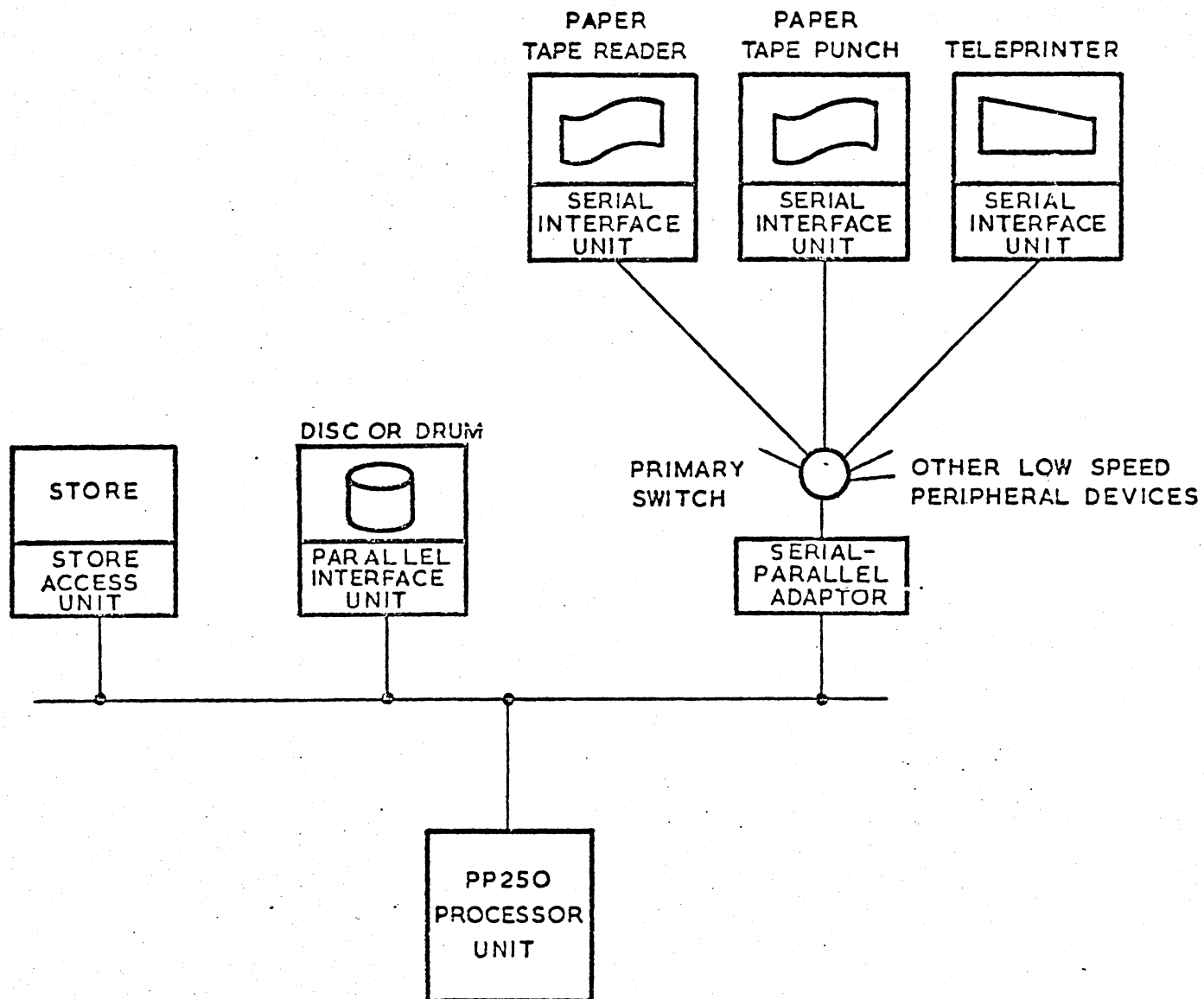
All information which is to be retained in the system must be named, directly or indirectly, and stored in the Directory structure. The Directory Control commands permit the user to store and retrieve such information and if necessary enables information to be passed from one user to another in a controlled manner. Typically, all background packages which are run on specific command from the operator would be recorded in a Directory as would all files and packages in a program development environment.

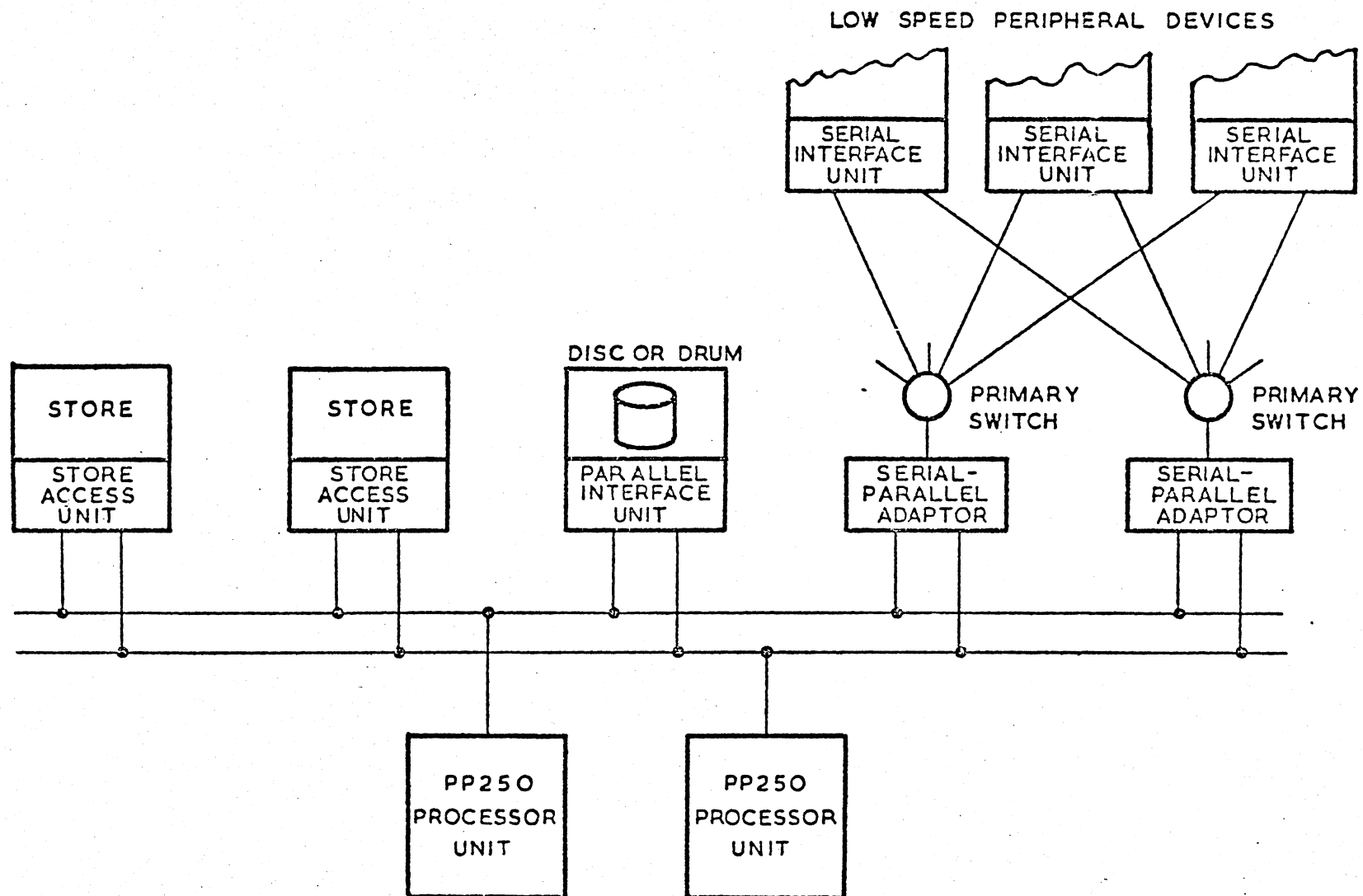
3.4.5. File Editing

A line editor is available which carries out alterations to text files. These files may contain programs under development, sequences of commands, or any other textual information. The facilities include the inclusion and deletion of lines of text, and commands for allocation, and de-allocation of files.

3.4.6. Store and Process Commands

These commands mirror the basic functions of the Store Manager and Process Manager. They permit the allocation of store, Flags and processes and also the various operations that can be carried out on them.





SMALL SECURE SYSTEM

FIG. 2

